

Pokémon Swim Championship

Informe Final — Proyecto de Videojuego

David Salomon Ortiz Romero

Junio 2026

1. Introducción

Pokémon Swim Championship es un videojuego 2D desarrollado en C++ con la plataforma Qt, como proyecto final del curso de Informática II de la Universidad de Antioquia, semestre 2026-1. El juego combina mecánicas deportivas de natación con el universo de la franquicia Pokémon, enmarcándose en la temática propuesta por el curso: un deporte en un universo especial.

El proyecto pone en práctica los fundamentos del curso: programación orientada a objetos, gestión de memoria dinámica y desarrollo de interfaces gráficas con Qt. Aunque la idea principal es sencilla, durante el desarrollo se implementaron diferentes físicas, un agente rival adaptativo y una separación entre la lógica y la interfaz para facilitar la organización del proyecto.

El videojuego está orientado a cualquier persona familiarizada con el universo Pokémon que quiera experimentar una competencia de natación arcade con un nivel de dificultad configurable y un rival que aprende de su desempeño.

2. Planteamiento del problema

El curso requiere el desarrollo de un videojuego que demuestre de forma práctica los conocimientos adquiridos en programación orientada a objetos, memoria dinámica y diseño de interfaces gráficas. El desafío consiste en construir un sistema que sea funcionalmente correcto, visualmente agradable y con un nivel de dificultad no trivial, dentro de los límites de conocimiento correspondientes a un segundo semestre universitario.

El juego debe incluir al menos dos niveles con dinámicas significativamente distintas, un agente autónomo con comportamiento inteligente, al menos tres modelos físicos parametrizables, y una separación arquitectónica clara entre capa lógica y capa de interfaz gráfica.

3. Definición general y objetivos

3.1 Descripción general

El jugador controla a Totodile en una competencia de natación de dos etapas. En el primer nivel, una carrera lateral en piscina contra Psyduck — un agente inteligente que aprende del rendimiento del jugador. En el segundo nivel, un recorrido cronometrado en río cenital con corrientes y remolinos que el jugador debe sortear.

3.2 Objetivos

- Implementar una arquitectura orientada a objetos con herencia propia, polimorfismo y composición.
- Modelar al menos tres físicas no triviales aplicadas a los elementos del juego.
- Desarrollar un agente inteligente con los cuatro componentes requeridos: percepción, razonamiento, acción y aprendizaje.
- Separar la capa lógica de la capa GUI de forma que la lógica no dependa de Qt.

- Ofrecer una experiencia jugable con dificultad configurable no trivial.

4. Especificación de requerimientos

4.1 Requerimientos funcionales

- El juego debe tener dos niveles con dinámicas diferentes: vista lateral (Nivel 1) y vista cenital (Nivel 2).
- El jugador puede moverse horizontalmente (A/D para bracear) y verticalmente (W/S) en el Nivel 1.
- El jugador puede activar la habilidad Mordedor (tecla E) que destruye obstáculos cercanos.
- Los Magikarps deben aparecer con trayectoria parabólica real que varía según la dificultad.
- Psyduck debe aprender el tiempo del jugador y ajustar su velocidad en cada carrera.
- El Nivel 2 debe tener un temporizador descendente y corrientes de río parametrizables.
- El juego debe reproducir música de fondo y dos efectos de sonido de eventos.
- El menú debe permitir seleccionar la dificultad antes de cada partida.

4.2 Requerimientos no funcionales

- La capa lógica no debe depender de ninguna clase de Qt.
- Toda memoria dinámica debe ser gestionada correctamente con new/delete.
- Las excepciones deben validar rangos en los constructores.
- El loop de juego debe correr a 60fps (timer de 16ms).
- Los sprites deben tener fondo transparente y escalar sin distorsión.

5. Metodología y planificación

El desarrollo siguió una metodología incremental organizada en los tres momentos establecidos por el curso. El Momento I definió la idea general, los niveles y la narrativa. El Momento II diseñó el diagrama de clases de la capa lógica y la arquitectura del agente. El Momento III implementó progresivamente el sistema completo.

Se realizaron varios commits durante el desarrollo para registrar los avances del proyecto: primero la estructura del proyecto Qt, luego las clases de lógica de menor

a mayor complejidad, después la GUI básica, y finalmente los sprites, sonido y ajustes de balance.

Las pruebas se realizaron manualmente ejecutando el juego en cada iteración, verificando el comportamiento de cada física, la colisión entre personajes y obstáculos, el ciclo del agente y el manejo de excepciones.

6. Diseño y arquitectura del sistema

6.1 Separación en capas

El sistema se divide en dos capas bien definidas. La capa lógica contiene toda la lógica del juego y no tiene ninguna dependencia de Qt. La capa GUI contiene las clases que heredan de Qt y se encargan exclusivamente del renderizado, la captura de input y la reproducción de sonido.

6.2 Capa lógica — jerarquía de clases

Personaje (abstracta): clase base de la jerarquía. Define los atributos comunes de posición, velocidad y estado, y declara el método virtual puro actualizar(dt). No puede instanciarse directamente.

Jugador: hereda de Personaje. Implementa la física de resistencia hidrodinámica $v(t) = v_0 \cdot e^{(-k \cdot t)}$ en actualizar(). Gestiona las habilidades Hiperactivo (burst de velocidad inicial de 3 segundos) y Mordedor (destruye obstáculos cercanos con penalización de velocidad). El coeficiente k varía con la dificultad seleccionada.

AgenteRival: hereda de Personaje. Implementa el ciclo del agente inteligente con el mismo modelo de decaimiento exponencial que el jugador, ejecutando braceadas automáticas a intervalos que dependen de su velocidadBase. Tras cada carrera hace el ciclo percibir → razonar → actuar → aprender.

Obstaculo: modela cualquier obstáculo dinámico del juego. Usa un arreglo float* (memoria dinámica) para almacenar los parámetros de su física. El tipo determina qué ecuación ejecuta: 0 = Magikarp (parábola), 1 = remolino (fuerza centrípeta), 2 = objeto estático.

Nivel: controla la lógica completa de un nivel en composición. Crea y destruye a Jugador, AgenteRival y el vector de Obstaculo*. Maneja las colisiones, la generación de obstáculos y las condiciones de fin.

Juego: controla el flujo global entre niveles y pantallas. Mantiene el estado actual (MENU, JUGANDO, RESULTADO, FIN) y la dificultad seleccionada.

6.3 Capa GUI

MainWindow: hereda de QMainWindow. Gestiona el QStackedWidget con tres pantallas: menú, juego y resultado. Construye los widgets del menú por código sin usar archivos .ui.

NivelWidget: hereda de QWidget. Implementa el loop de juego mediante un QTimer de 16ms conectado al slot tick() con la sintaxis de puntero a función de Qt5+. En paintEvent() dibuja el nivel usando QPainter. Maneja el sonido con QSoundEffect (efectos WAV) y QMediaPlayer (música MP3).

6.4 Relaciones entre clases

Desde	Hacia	Tipo	Multiplicidad
-------	-------	------	---------------

Jugador	Personaje	Herencia	— (herencia propia)
AgenteRival	Personaje	Herencia	— (herencia propia)
Juego	Nivel	Composición	1 Juego → 1 Nivel
Nivel	Jugador	Composición	1 Nivel → 1 Jugador
Nivel	AgenteRival	Composición	1 Nivel → 1 AgenteRival
Nivel	Obstaculo	Composición	1 Nivel → 0..* Obstaculo
MainWindow	NivelWidget	Creación	1 MainWindow → 1 NivelWidget
NivelWidget	Juego*	Uso (puntero)	NivelWidget accede a Juego vía puntero

7. Desarrollo e implementación

7.1 Modelos físicos

El juego utiliza tres modelos físicos diferentes, cuyos parámetros pueden modificarse para ajustar la dificultad y el comportamiento de cada nivel.

Resistencia hidrodinámica (Nivel 1): cuando el jugador deja de bracear, su velocidad disminuye de forma gradual siguiendo un modelo de decaimiento exponencial. Para representar distintos niveles de dificultad, el coeficiente (k) toma los valores 0.8, 1.4 y 2.2 en fácil, normal y difícil, respectivamente. En la práctica, esto hace que en las dificultades más altas sea necesario mantener un ritmo de braceo más constante para no perder mucha velocidad.

Movimiento parabólico (Nivel 1): los Magikarps que aparecen como obstáculos siguen trayectorias parabólicas. La velocidad inicial se calcula de manera que cada salto alcance una altura aleatoria dentro de un rango establecido, lo que evita que todos los obstáculos se comporten exactamente igual. Dependiendo de la trayectoria generada, el tiempo que permanecen en el aire suele estar entre 2 y 3 segundos.

Corriente oscilatoria (Nivel 2): en el río existe una corriente lateral que cambia continuamente con el tiempo, obligando al jugador a corregir su trayectoria mientras avanza. Además, los remolinos generan una fuerza de atracción hacia su centro, haciendo que Totodile sea arrastrado si se acerca demasiado. Ambos efectos aumentan la dificultad de controlar el movimiento y completar el recorrido dentro del tiempo límite.

7.2 Agente inteligente — Psyduck

Psyduck implementa un agente de aprendizaje basado en historial de tiempos con los cuatro componentes requeridos:

- **Percepción:** al terminar cada carrera, el sistema llama a `percibir(tiempoJugador)` con el tiempo total de esa carrera en segundos.
- **Razonamiento:** `razonar()` calcula el promedio de las últimas 3 carreras del `historialTiempos` (`std::vector<float>`). Si el jugador fue más rápido que el promedio, la diferencia es negativa y el ajuste resultante aumenta `velocidadBase`.
- **Acción:** `actuar()` aplica el ajuste a `velocidadBase` con `std::clamp` entre 70 y 200. También recalcula k para que Psyduck tenga un decaimiento coherente con su velocidad.
- **Aprendizaje:** `aprender()` hace `push_back` del tiempo más reciente al historial, que persiste durante toda la sesión. A mayor historial, el promedio es más representativo del nivel real del jugador.

Psyduck no avanza en línea recta sino que ejecuta braceadas automáticas con el mismo modelo de decaimiento que el jugador, con un intervalo entre braceadas inversamente proporcional a su velocidadBase. Esto evita que el rival avance a una velocidad completamente constante.

7.3 Memoria dinámica y excepciones

La memoria dinámica se usa en tres lugares del sistema. En Obstaculo, el arreglo params es un float* reservado con new[] en el constructor y liberado con delete[] en el destructor. El constructor de copia realiza una copia profunda del arreglo para garantizar independencia entre instancias. En Nivel, Jugador y AgenteRival se crean con new y se destruyen en el destructor de Nivel. En Juego, el Nivel actual se crea y destruye con new/delete al cambiar de nivel.

Las excepciones de tipo std::invalid_argument se lanzan en los constructores de Jugador (dificultad fuera de 1-3), AgenteRival (tiempo negativo en percibir), Obstaculo (params nulo, nParams<4, tipo inválido), Nivel (número de nivel o dificultad inválidos) y Juego (número de nivel inválido en iniciarNivel). Todas se capturan en MainWindow con try-catch y se muestran al usuario con QMessageBox.

7.4 Contenedores STL

Contenedor	Clase	Justificación
std::vector<Obstaculo*>	Nivel	Colección de tamaño variable de obstáculos activos. Crece con cada generación y se reduce con limpiarObstaculos(). El acceso por índice es O(1) y la inserción al final O(1) amortizado.
std::vector<float>	AgenteRival	Historial de tiempos del jugador. Crece una vez por carrera y se recorre secuencialmente para calcular el promedio. Su tamaño es impredecible en tiempo de compilación.

8. Procedimientos de prueba

8.1 Pruebas de lógica

Se realizaron pruebas unitarias en C++ puro (sin Qt) sobre las clases de la capa lógica, verificando los siguientes aspectos:

- Validación de rangos: dificultad 0, -1 y 4 lanzan `std::invalid_argument` en el constructor de Jugador. Dificultades 1, 2 y 3 son aceptadas.
- Física de decaimiento: tras un braceo, la velocidad decrece monotónicamente sin bracear adicional y nunca toma valores negativos.
- Hiperactivo: activo al inicio, se desactiva después de 3 segundos simulados.
- Mordedor: se activa, se desactiva después de 2 segundos, no se puede activar dos veces seguidas.
- Stamina: inicia en 100, baja al bracear, nunca es negativa, se recupera con el tiempo, no supera 100.
- Agente: acelera cuando el jugador mejora, se relaja cuando empeora, respeta los límites de 70 y 200.
- Obstaculo: lanza excepciones con params nulo o $nParams < 4$, la Y cambia con la parábola, el constructor de copia es independiente del original.

8.2 Pruebas de integración

Las pruebas de integración se realizaron ejecutando el juego completo y verificando:

- Los Magikarps aparecen delante del jugador y llegan hasta la zona donde este se encuentra.
- La colisión reduce la velocidad del jugador correctamente.
- El Mordedor destruye Magikarps y el sonido de mordisco se reproduce en el momento de activación.
- El sonido de splash se reproduce al recibir un golpe.
- La barra de progreso refleja correctamente la posición de ambos competidores.
- El agente ajusta su velocidad en la segunda carrera respecto a la primera.
- El timer del Nivel 2 llega a cero y termina el nivel correctamente.

9. Guía de instalación y uso

9.1 Requisitos técnicos

- Qt 6.x con módulos QtWidgets y QtMultimedia instalados.
- Compilador g++ con soporte C++17.

- Sistema operativo: Windows 10/11 o Linux (Ubuntu 22+).

9.2 Compilación desde código fuente

Abrir Qt Creator, cargar el archivo `pokemon_swim.pro`, seleccionar el kit de compilación apropiado, cambiar a modo Release y compilar con Ctrl+B. El ejecutable se genera en la carpeta de build.

9.3 Ejecución del ejecutable

En Windows, ejecutar `pokemon_swim.exe` desde la carpeta `build/` que contiene el ejecutable y todas las DLLs de Qt generadas por `windeployqt`. En Linux, ejecutar `./pokemon_swim` desde la carpeta de build.

9.4 Controles

Tecla	Nivel 1	Nivel 2
A / D	Bracear (alternando para avanzar)	Moverse izquierda / derecha
W / S	Subir / bajar en el carril	Moverse arriba / abajo
E	Activar Mordedor	Activar Mordedor

10. Resultados y discusión

El juego funciona de forma estable en los dos niveles. La física de decaimiento exponencial genera una sensación natural de resistencia del agua que se diferencia claramente entre niveles de dificultad. Los Magikarps representan una amenaza real ya que su apex aleatorio cubre toda la zona de movimiento vertical del jugador, requiriendo atención constante.

El agente Psyduck funciona como rival adaptativo: en las primeras carreras es relativamente predecible, pero con tres o más carreras jugadas su velocidadBase converge al nivel del jugador, haciendo que cada partida sea progresivamente más competitiva. Uno de los aspectos que más tiempo tomó fue ajustar los parámetros del agente para que no fuera demasiado fácil ni demasiado difícil.

La separación en capas funcionó correctamente: en ningún momento fue necesario incluir un header de Qt en las clases de lógica, lo que facilita las pruebas unitarias y mantiene la arquitectura limpia.

Las limitaciones actuales incluyen que el Nivel 2 no tiene un rival visible, lo que hace la experiencia menos competitiva que el Nivel 1. En una versión futura se podría agregar un indicador de progreso de Psyduck también en el Nivel 2. La física de los remolinos podría refinarse para que la atracción espiral sea más orgánica.

11. Cambios respecto a los momentos anteriores

11.1 Cambios desde el Momento I

- Se agregó movimiento vertical del jugador en el Nivel 1 (teclas W/S), que no estaba en la propuesta original. Esto hizo los Magikarps más relevantes como obstáculo.
- El agente Psyduck en el Momento I se describía como un rival que simplemente avanzaba. La implementación final hace braceadas propias con el mismo modelo de decaimiento que el jugador.
- La habilidad Hiperactivo se mantuvo como burst inicial de 3 segundos, pero se hace más notable en dificultad difícil porque el decaimiento posterior es más pronunciado.

11.2 Cambios desde el Momento II

- El diagrama de clases del Momento II no incluía los atributos jugadorCarrilY ni timerMuerte en Obstaculo, que se agregaron durante la implementación para gestionar el movimiento vertical y la animación post-colisión.
- Se eliminó la clase ModeloFisico que se había considerado: las físicas quedaron como métodos internos de Jugador, AgenteRival y Obstaculo, evitando una clase sin identidad propia.
- Se agregaron los tiles de piscina y río como sprites individuales cargados desde el sistema de recursos de Qt (.qrc), lo que no estaba contemplado en el diseño inicial.

- El agente incorporó el atributo `timerBraceada` para las braceadas automáticas, que no aparecía en el diagrama del Momento II.

12. Conclusiones

El desarrollo de Pokémon Swim Championship permitió poner en práctica de forma integrada todos los temas del curso. La herencia y el polimorfismo se usaron de manera natural con la jerarquía Personaje → Jugador / AgenteRival, donde el método virtual `puro actualizar()` garantiza que cada entidad defina su propio comportamiento sin acoplamiento con las otras.

La parte que presentó más dificultades fue el manejo de memoria dinámica, especialmente en las clases que utilizaban memoria reservada manualmente. El uso de `new/delete` con constructores de copia correctos en `Obstaculo` evitó fugas de memoria y dobles liberaciones. La regla de tres se respetó en todas las clases que gestionan recursos propios.

El agente inteligente resultó ser la parte más interesante: un sistema simple de promedio histórico genera un comportamiento que el jugador percibe como adaptativo, lo que demuestra que la inteligencia artificial de juegos no necesita ser compleja para ser efectiva.

La separación entre capa lógica y capa GUI demostró ser útil durante las pruebas: fue posible probar la física y el agente completamente en C++ puro sin necesitar Qt, lo que aceleró significativamente la detección de errores.

13. Referencias

- Documentación oficial de Qt 6: <https://doc.qt.io/qt-6/>
- The Spriters Resource — sprites de Pokémon Gen II: <https://www.spriters-resource.com>
- OpenGameArt.org — música y efectos de sonido libres de derechos.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- Especificación del Proyecto Final 2026-1, Universidad de Antioquia, Informática II.